RESEARCHHUB AI

Intelligent Research Paper Management and Analysis System using Agentic AI

React + TypeScript | FastAPI | Groq Llama 3.3 70B | SQLAlchemy

Project Documentation

Table of Contents

Project Description 3
How It Works 5
Prior Knowledge 5
Project Workflow 6
Milestone 1: Requirements Specification and Project Setup 7
Milestone 2: Groq API Integration and Model Initialization 8
Milestone 3: Backend Development with FastAPI 9
Milestone 4: Frontend Development with React and TypeScript 11
Milestone 5: AI Agent Implementation and Context Management 11
Milestone 6: Testing and Deployment 13
Project Structure 14
UI Walkthrough 15
Conclusion 20

Project Description

With the exponential growth of academic research publications and the increasing complexity of staying current with scientific literature, researchers often struggle to efficiently discover, organize, and analyze relevant research papers. Traditional methods of paper management involve manual searching, downloading, organizing, and reading through extensive literature — a time-consuming process that limits research productivity.

To address this challenge, ResearchHub AI was developed as an intelligent, agentic AI-powered research paper management platform. Built using React and TypeScript for the frontend, FastAPI for backend processing, and integrated with Groq's Llama 3.3 70B model for advanced natural language understanding, the platform enables researchers to seamlessly search for papers, import them into personal workspaces, and interact with an AI chatbot that provides contextual insights, summaries, and answers based on the research content.

Scenario 1: Efficient Research Paper Discovery and Management

Researchers query academic databases (arXiv) through an intelligent search interface, receiving curated results with metadata (title, authors, date, abstract). Papers are imported into personal workspaces with a single click. The React frontend provides an intuitive browsing experience while the FastAPI backend handles database APIs, response processing, and concurrent user management seamlessly.

Scenario 2: AI-Powered Contextual Research Analysis

The chatbot, powered by Groq's Llama 3.3 70B, answers research-specific questions by accessing paper content directly. Researchers can ask “What are the key differences between the two papers?” or “Summarize the main findings across papers.” The AI synthesizes information across multiple documents, reducing manual reading time and enabling rapid comprehension of complex topics.

Scenario 3: Organized Workspace Collaboration and Knowledge Management

Researchers create multiple workspaces for different projects — e.g., “Machine Learning” and “Medical Imaging.” The AI chatbot maintains context-specific conversations, remembering queries and building refined insights. The system stores conversation history and paper relationships, enabling exploration tracking. FastAPI ensures efficient data management while JWT-based authentication protects sensitive research data and maintains privacy.

Architecture

Frontend Layer	React + TypeScript — Login | SearchPapers | Dashboard | Workspace | AITools | DocSpace
API Layer	FastAPI Backend — Auth Router | Papers Router | Chat Router | AI Tools Router | Workspaces Router | Docs Router
AI Layer	Groq API → Llama 3.3 70B (Versatile)
Data Layer	SQLAlchemy ORM → SQLite / PostgreSQL
External Layer	arXiv REST API

Pre-requisites

- **FastAPI Framework Knowledge**: FastAPI Documentation
- **Groq API Familiarity**: https://console.groq.com/
- **HTML, CSS, and JavaScript Skills**: W3Schools HTML/CSS/JavaScript Tutorials
- **Python Programming Proficiency**: Python Documentation
- **Version Control with Git**: Git Documentation
- **Development Environment Setup**: FastAPI Installation Guide

Key Features

- User Registration & Login
- Research Paper Search & Import (arXiv)
- Workspace Management
- AI Chatbot with Context Awareness
- AI Summaries, Key Insights & Literature Review generation
- PDF Upload, Text Extraction & AI Summary
- DocSpace rich-text note editor
- Conversation History

How It Works

Step 1: Users register and authenticate through a secure login system powered by JWT tokens for session management and security.

Step 2: Researchers search for academic papers using keyword-based queries sent to the FastAPI backend API endpoints.

Step 3: The system queries the arXiv academic database and returns relevant papers with comprehensive metadata including abstracts, citations, and publication details.

Step 4: Users can import selected papers into their personal workspaces for organization, categorization, and future reference with one-click functionality.

Step 5: The AI chatbot, powered by the Groq Llama 3.3 70B model, analyzes the papers in a workspace and maintains contextual understanding across conversations.

Step 6: Users interact with the chatbot to get summaries, insights, comparisons, and research-specific answers derived from their curated paper collection.

Step 7: Built with a React/TypeScript frontend and FastAPI backend, the platform ensures real-time responsiveness, secure data handling, and intelligent conversational AI assistance for enhanced research productivity.

Prior Knowledge

You must have prior knowledge of the following topics to work with this project:

Agentic AI and Groq API

An agentic AI system represents advanced artificial intelligence that can autonomously plan, reason, and execute complex tasks without constant human supervision. In this project, the Llama 3.3 70B model integrated with the Groq API enables ultra-fast inference for real-time research paper analysis and contextual conversation.

FastAPI

Knowledge of building high-performance web backends using FastAPI, a modern Python framework used for developing RESTful APIs. FastAPI's automatic API documentation, async support, and type hints make it ideal for AI-powered applications requiring real-time responses.

Frontend Development (React, TypeScript, Tailwind CSS)

Familiarity with creating responsive, interactive web applications using React with TypeScript for type safety and a better development experience. Understanding component architecture, state management, API integration, and modern styling with Tailwind CSS is required. Knowledge of building conversational interfaces, search components, and workspace management UIs is particularly important.

Databases & Semantic Retrieval

Basic understanding of relational databases (SQLAlchemy models) and how paper metadata, workspaces, and conversation history are related and retrieved to build context for the AI chatbot.

Project Workflow

Milestone 1: Requirements Specification and Project Setup

- Activity 1.1: Create requirements.txt file
- Activity 1.2: Install the required libraries

Milestone 2: Groq API Integration and Model Initialization

- Activity 2.1: Generate Groq API Key
- Activity 2.2: Configure API credentials
- Activity 2.3: Initialize the Groq client and model

Milestone 3: Backend Development with FastAPI

- Activity 3.1: Create authentication endpoints
- Activity 3.2: Implement paper search API
- Activity 3.3: Build AI chatbot endpoints

Milestone 4: Frontend Development with React and TypeScript

- Activity 4.1: Create authentication components and pages

Milestone 5: AI Agent Implementation and Context Management

- Activity 5.1: Develop research paper analysis functions

Milestone 6: Testing and Deployment

- Activity 6.1: Running the Backend
- Activity 6.2: Running the Frontend
- Activity 6.3: Configure CORS and environment variables

Milestone 1: Requirements Specification and Project Setup

This milestone focused on defining the project's objectives, functional requirements, and technical specifications. It involved selecting the necessary tools, frameworks, and libraries while setting up the development environment for smooth implementation.

Activity 1.1: Create requirements.txt file

```
fastapi>=0.104.0
uvicorn[standard]>=0.24.0
python-dotenv>=1.0.0
groq>=0.4.0
httpx>=0.25.0
python-multipart>=0.0.6
python-jose[cryptography]>=3.3.0
passlib[bcrypt]>=1.7.4
bcrypt>=4.0.0
sqlalchemy>=2.0.0
pypdf>=3.17.0
numpy>=1.26.0
```

Activity 1.2: Install the required libraries

```
# Create virtual environment
python -m venv venv
source venv/bin/activate
# On Windows: venv\Scripts\activate

# Install dependencies
cd backend
pip install -r requirements.txt
```

Milestone 2: Groq API Integration and Model Initialization

This milestone focused on integrating the Groq API to enable high-speed AI inference and response generation within the application.

Activity 2.1: Generate Groq API Key

- Visit https://console.groq.com/
- Click on API Keys from the navigation menu
- Click on Create API Key
- Copy the generated API key
- Store it securely in your environment variables

Activity 2.2: Configure API credentials

Create a .env file in the backend directory:

```
GROQ_API_KEY=gsk_your_groq_api_key_here
SECRET_KEY=your_jwt_secret_key_here
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=1440
DATABASE_URL=sqlite:///./researchhub.db
```

Activity 2.3: Initialize the Groq client and model

```
# backend/utils/groq_client.py
import os
from groq import Groq
from dotenv import load_dotenv

load_dotenv()
client = Groq(api_key=os.getenv("GROQ_API_KEY"))

MODEL_CONFIG = {
    "model": "llama-3.3-70b-versatile",
    "temperature": 0.3,
    "max_tokens": 2000,
    "top_p": 0.9
}
```

Milestone 3: Backend Development with FastAPI

This milestone focused on developing the core backend infrastructure using FastAPI to manage requests, responses, and data flow efficiently.

Activity 3.1: Create authentication endpoints

```
# backend/routers/auth.py
router = APIRouter(prefix="/api/auth")

@router.post("/register")
def register(user_data: UserCreate, db: Session = Depends(get_db)):
    hashed_password = pwd_context.hash(user_data.password)
    return {"message": "User registered successfully"}

@router.post("/login")
def login(user_credentials: UserLogin, db: Session = Depends(get_db)):
    access_token = create_access_token(data={"sub": user.email})
    return {"access_token": access_token, "token_type": "bearer"}

@router.get("/me")
def get_me(current_user: User = Depends(get_current_user)):
    return current_user
```

Activity 3.2: Implement paper search API

```
# backend/routers/papers.py
router = APIRouter(prefix="/api/papers")

@router.get("/search")
async def search_papers(query: str, current_user: User = Depends(get_current_user)):
    results = await query_arxiv(query)
    return {"papers": results}

@router.post("/import")
def import_paper(paper_data: PaperImport, current_user: User =
Depends(get_current_user)):
    imported_paper = store_paper(paper_data, current_user.id)
    return {"message": "Paper imported successfully", "paper": imported_paper}

@router.post("/upload")
def upload_pdf(file: UploadFile, current_user: User = Depends(get_current_user)):
    text = extract_pdf_text(file)
    return {"extracted_text": text}
```

Activity 3.3: Build AI chatbot endpoints

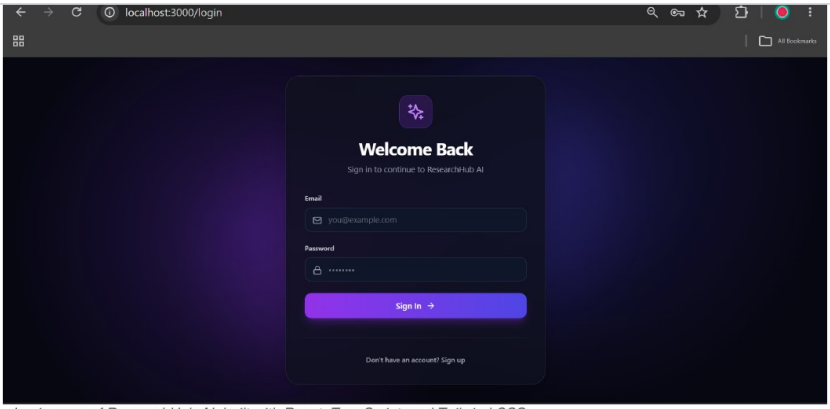
```
# backend/routers/chat.py
router = APIRouter(prefix="/api/chat")

@router.post("")
def chat_with_papers(message: ChatMessage, workspace_id: int,
                     current_user: User = Depends(get_current_user), db: Session =
Depends(get_db)):
    workspace_papers = get_workspace_papers(workspace_id, current_user.id)
    context = create_research_context(workspace_papers, message.content)
    response = generate_chat_response(
        [{"role": "user", "content": message.content}], context_str=context)
    store_conversation(workspace_id, message.content, response)
    return {"response": response}
```

Milestone 4: Frontend Development with React and TypeScript

This milestone focused on building an interactive and responsive user interface using React and TypeScript. It involved developing dynamic components for user interaction, integrating API calls for real-time data exchange, and implementing a clean, modular design for better maintainability. The goal was to deliver a seamless, visually appealing frontend that enhances user experience while ensuring type safety and performance efficiency.

Frontend login component with state management and form handling.



Login page of ResearchHub AI, built with React, TypeScript, and Tailwind CSS.

Milestone 5: AI Agent Implementation and Context Management

This milestone focused on implementing the AI agent to handle intelligent interactions and maintain contextual

Login page of ResearchHub AI, built with React, TypeScript, and Tailwind CSS.

Milestone 5: AI Agent Implementation and Context Management

This milestone focused on implementing the AI agent to handle intelligent interactions and maintain contextual awareness across user sessions. It involved designing logic for multi-turn conversations, context retention, and dynamic response adaptation.

Activity 5.1: Develop research paper analysis functions

The create_research_context method aggregates paper information (titles, authors, abstracts) into a structured prompt that provides the LLM with relevant background information. The generate_chat_response method sends this context to the Groq API, enabling the AI agent to provide informed, contextually-accurate answers based on the researcher's imported papers. This architecture enables autonomous reasoning and synthesis across multiple research documents.

```
# backend/routers/ai_tools.py
router = APIRouter(prefix="/api/ai")

@router.post("/summarize")
def generate_summaries(req: AIToolRequest, current_user: User =
Depends(get_current_user),
                       db: Session = Depends(get_db)):
    papers = db.query(Paper).filter(Paper.id.in_(req.paper_ids)).all()
    summaries = []
    for paper in papers:
        context = f"Title: {paper.title}\nAuthors: {paper.authors}\nAbstract:
{paper.abstract}"
        prompt = "Provide a comprehensive concise summary in 5-7 bullet points..."
        summary_text = generate_chat_response(
            [{"role": "user", "content": prompt}], context_str=context)
        summaries.append({"paper_id": paper.id, "title": paper.title, "summary":
summary_text})
    return {"results": summaries}

@router.post("/insights")
def extract_insights(req: AIToolRequest, ...): ...

@router.post("/literature-review")
def generate_literature_review(req: AIToolRequest, ...): ...
```

Milestone 6: Testing and Deployment

This milestone focused on verifying the overall functionality, performance, and reliability of the system before deployment. Comprehensive testing was conducted to ensure all components worked cohesively. After successful testing, the application was deployed in a configured environment, ensuring smooth execution, scalability, and readiness for real-world use.

Activity 6.1: Running the Backend

```
cd backend
source venv/bin/activate
# On Windows: venv\Scripts\activate
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Activity 6.2: Running the Frontend

```
cd frontend
npm install
npm run dev
# or
npm start
```

Activity 6.3: Configure CORS and environment variables

```
# backend/main.py
app = FastAPI(title="ResearchHub AI API", version="1.0.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000", "http://localhost:5173", "*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/")
async def root():
    return {"status": "online", "system": "ResearchHub AI API", "version": "1.0.0"}
```

Project Structure

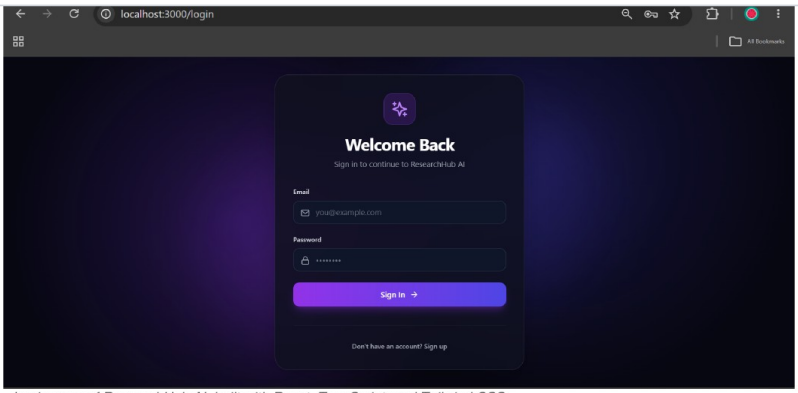
Create the ResearchHub AI project folder with the following structure:

```
Research_Pilot/
|-- backend/
|   |-- main.py
|   |-- database.py
|   |-- requirements.txt
|   |-- .env.example
|   |-- models/
|   |   |-- user.py
|   |   |-- workspace.py
|   |   |-- paper.py
|   |   |-- chat.py
|   |   `-- doc.py
|   |-- routers/
|   |   |-- auth.py
|   |   |-- workspaces.py
|   |   |-- papers.py
|   |   |-- chat.py
|   |   |-- ai_tools.py
|   |   `-- docs.py
|   `-- utils/
|       |-- arxiv_client.py
|       |-- paper_search.py
|       |-- pdf_parser.py
|       |-- embeddings.py
|       `-- groq_client.py
|-- frontend/
|   |-- src/
|   |   |-- components/
|   |   |   |-- Navbar.tsx
|   |   |   |-- Sidebar.tsx
|   |   |   `-- Modal.tsx
|   |   |-- pages/
|   |   |   |-- Home.tsx
|   |   |   |-- Login.tsx
|   |   |   |-- Dashboard.tsx
|   |   |   |-- SearchPapers.tsx
|   |   |   |-- WorkspaceDetail.tsx
|   |   |   |-- AITools.tsx
|   |   |   |-- UploadPDF.tsx
|   |   |   `-- DocSpace.tsx
|   |   |-- utils/api.ts
|   |   `-- App.tsx
|   |-- package.json
|   `-- tailwind.config.js
`-- README.md
```

UI Walkthrough

Login Page

The login page serves as the primary entry point for researchers to access the ResearchHub AI platform. Users can enter their credentials securely, with form validation ensuring proper email and password format. This authentication gateway protects user research data and ensures each researcher's papers and conversations remain private and organized within their personal workspace.



Login page of ResearchHub AI, built with React, TypeScript, and Tailwind CSS.

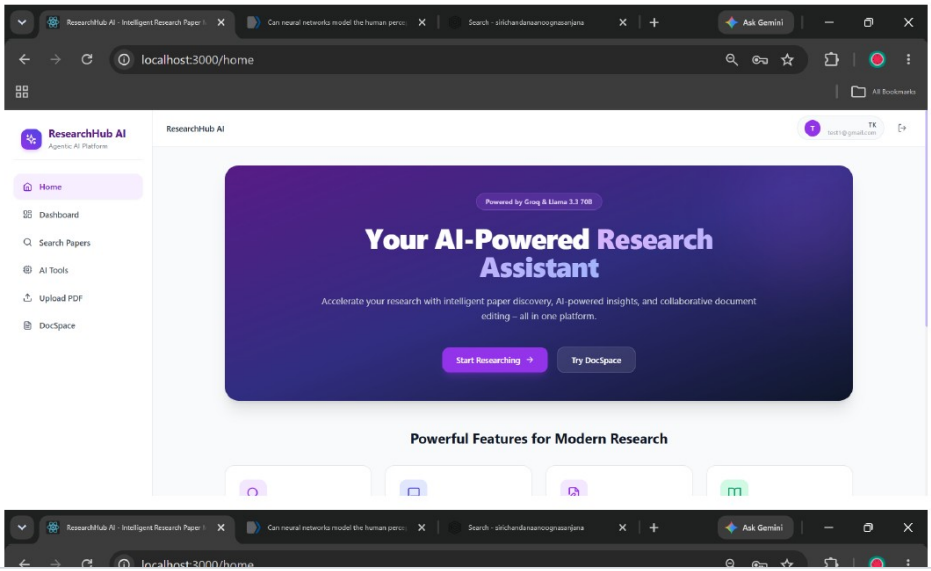
Milestone 5: AI Agent Implementation and Context Management

This milestone focused on implementing the AI agent to handle intelligent interactions and maintain contextual

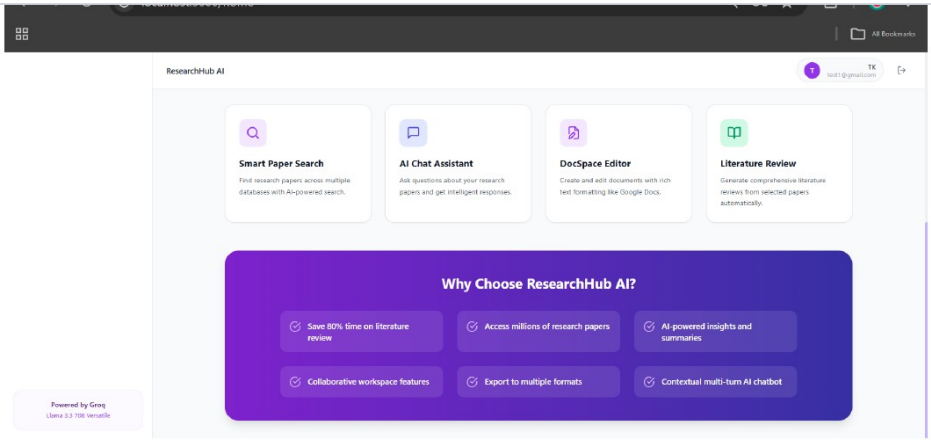
Login page — secure sign-in with email and password.

Home Page

The home page provides an overview of the platform and its key features, welcoming new and returning users. It displays navigation options to access paper search functionality, existing workspaces, and the AI chatbot interface. This landing area helps users understand the platform's capabilities and quickly navigate to the features they need for their research tasks.



Home page — hero section introducing the AI-powered research assistant.

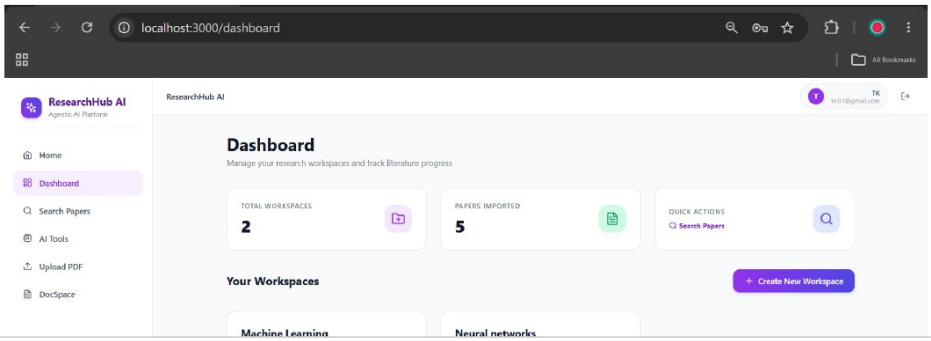


Home page — feature highlights and platform benefits.

Dashboard

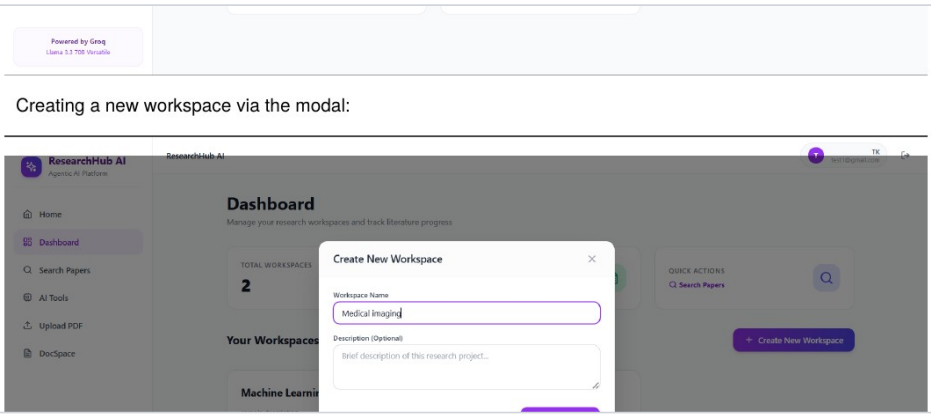
The dashboard offers a personalized view of the researcher's activity and research progress. Users can see total workspaces, papers imported, and quick links to frequently used features. This central hub enables researchers to efficiently manage their research projects and track their analysis progress across multiple ongoing studies.

researchers to efficiently manage their research projects and track their analysis progress across multiple ongoing studies.

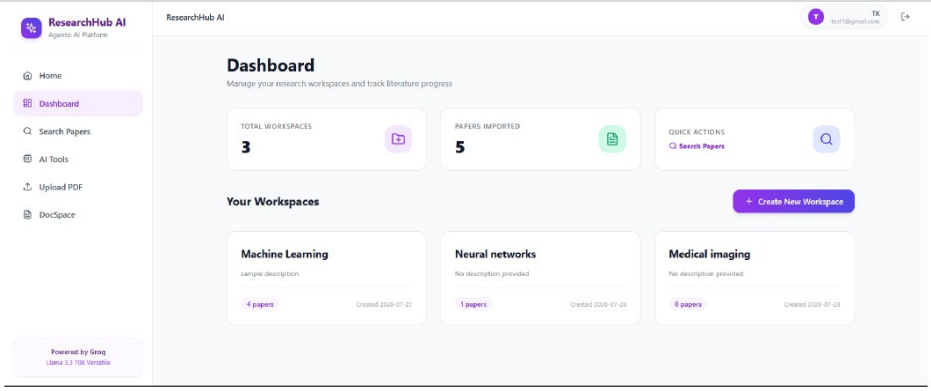


Dashboard — workspace overview and quick actions.

Creating a new workspace via the modal:



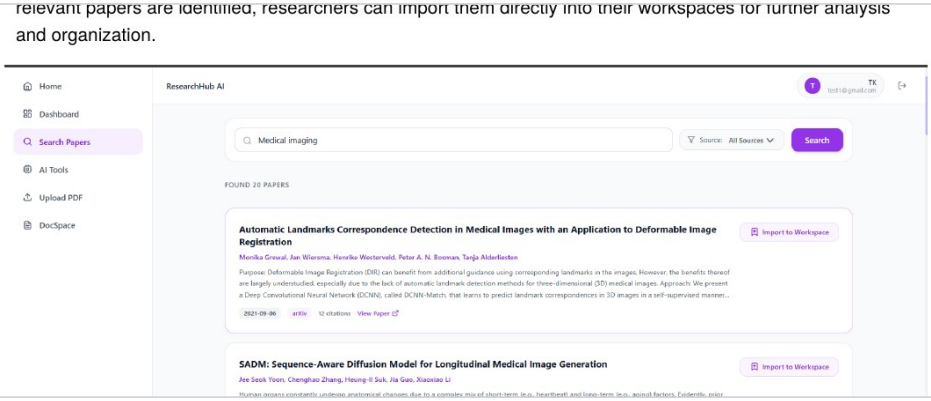
Creating a new workspace via the modal.



Dashboard after the new workspace has been created.

Search Papers Page

The search papers page enables researchers to query academic databases for relevant publications. Users can enter keywords or research topics and preview paper metadata including abstracts and citations. Once relevant papers are identified, researchers can import them directly into their workspaces for further analysis and organization.



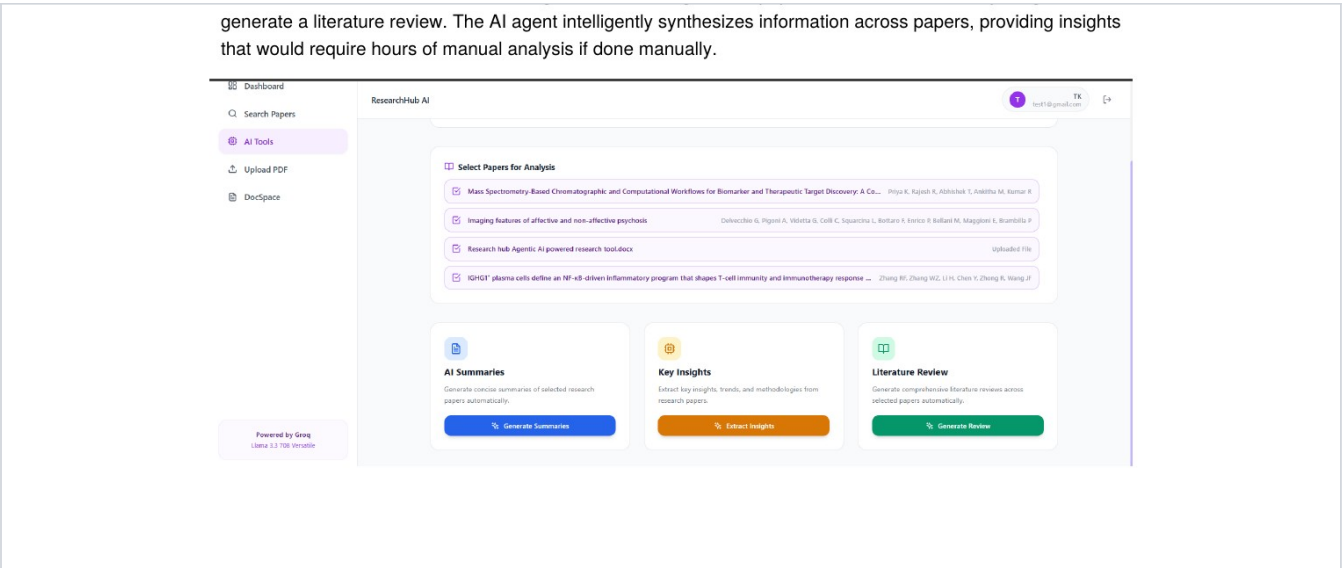
Search papers page — keyword search across academic databases.



Viewing the full paper via the source (arXiv).

AI Tools

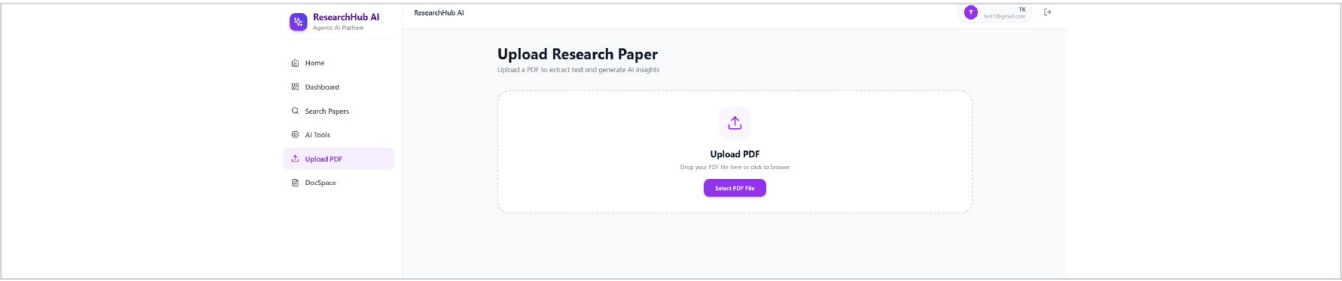
The AI Tools section provides access to advanced research analysis capabilities powered by the Groq Llama 3.3 70B model. Researchers can leverage these tools to generate paper summaries, extract key insights, and generate a literature review. The AI agent intelligently synthesizes information across papers, providing insights that would require hours of manual analysis if done manually.



AI Tools — summaries, key insights, and literature review generation.

Upload PDF

The upload PDF functionality allows researchers to add papers from their personal collection into the system. Users can drag-and-drop or browse for PDF files, which are then processed and stored in their workspace. This feature enables researchers to complement database searches with papers they already possess, creating a comprehensive personal research library within the platform.

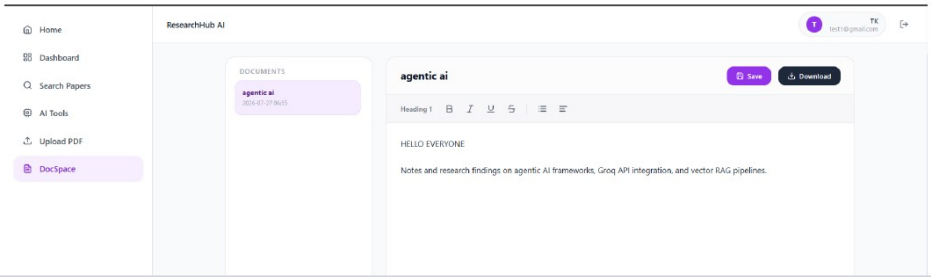


Upload PDF — drag-and-drop research paper ingestion.

DocSpace

The DocSpace section provides centralized access to all documents associated with a researcher's projects. This includes generated summaries, extracted notes, and AI-generated analysis reports. Researchers can organize documents by project and reference them easily during research and writing activities.

organize documents by project and reference them easily during research and writing activities.



DocSpace — rich-text editor for research notes and reports.

Conclusion

ResearchHub AI represents a significant advancement in research paper management through the integration of agentic AI technology. By combining React and TypeScript frontend development, FastAPI backend architecture, and Groq's Llama 3.3 70B model, the platform delivers an intelligent solution that automates paper discovery, enabling researchers to focus on insights rather than information management.

The agentic AI architecture enables autonomous reasoning and context-aware analysis, transforming how researchers interact with academic literature and extract meaningful knowledge from complex research domains. ResearchHub AI exemplifies how agentic AI systems can revolutionize knowledge work by automating tedious tasks and empowering scholars to achieve deeper research insights at scale.

Key Achievements

- Autonomous research assistance that independently analyzes papers and synthesizes insights
- One-click arXiv paper import
- Multi-workspace organization enabling parallel research projects
- A scalable FastAPI + React architecture supporting concurrent users